

MBAT Club Datathon 2026 — Participant Manual

MBAT Club

Table of contents

Welcome	5
How to use this manual	5
Before you arrive	5
Getting help on the day	6
1 Git + GitHub Desktop	7
1.1 The daily loop	7
1.2 Install (before the event)	7
1.3 Get your team's repository	7
1.4 Work session workflow	8
1.5 Avoid stepping on your teammates	8
1.6 Fix a merge conflict	8
1.7 Never commit these	9
1.8 Prefer the command line?	9
1.9 Before you submit (Day 2, 11:00 AM deadline)	9
2 Quarto	10
2.1 Anatomy of a .qmd file	10
2.2 Render your report	11
2.3 Set your output format	11
2.4 Build presentation slides (required from every team)	11
2.5 Set up PDF rendering (before the event)	12
2.6 Keep the report reproducible and inside the word limit	12
2.7 Learn more	13
3 Using the Starter Template	14
3.1 Folder structure	14
3.2 Get set up	14
3.3 Use your script from a .qmd	15
3.4 Fill in the report and slides	15
3.5 Render everything	15
3.6 Submit	15

I	Databases + MongoDB	16
4	NoSQL and MongoDB: An Introduction for Statisticians	17
4.1	1. Why This Tutorial Exists	17
4.2	2. What “NoSQL” Actually Means	18
4.3	3. The Core Idea: Documents Instead of Rows	18
4.3.1	A translation table for vocabulary	19
4.4	4. Why Embed Instead of Normalize?	19
4.5	5. Schema Flexibility: A Blessing and a Curse	20
4.6	6. Querying: <code>dplyr</code> /SQL Habits vs. MongoDB’s Query Language	21
4.6.1	Comparison table	21
4.7	7. Using MongoDB from R	22
4.8	8. Consistency, Transactions, and Why This Matters to You	23
4.9	9. When Each Approach Tends to Fit Better	23
4.10	10. A Worked Mini-Example: Same Data, Both Ways	24
4.11	11. Summary	25
5	Getting Your Competition Data Out of MongoDB Atlas and Into R	26
5.1	1. What you’ll need	26
5.2	2. Know your collections before you connect	27
5.3	3. Connect to the cluster	28
5.4	4. Pulling data out	29
5.4.1	4.1 Grab the fields you actually need	29
5.4.2	4.2 Filter server-side	29
5.4.3	4.3 Pull the incidents and join on <code>report_number</code>	30
5.4.4	4.4 Aggregation pipelines (optional, for heavier lifting)	30
5.4.5	4.5 Other nested/array fields	31
5.5	5. Cleaning the text	31
5.6	6. Tokenising with <code>tidytext</code>	32
5.6.1	6.1 Word frequencies	32
5.6.2	6.2 TF-IDF (which words distinguish one incident/domain from another)	33
5.6.3	6.3 N-grams	33
5.7	7. Sentiment analysis	34
5.8	8. Word co-occurrence / correlation (optional, <code>widyr</code>)	34
5.9	9. Putting it together: a minimal end-to-end script	35
5.10	10. Common pitfalls	35
5.11	11. Quick reference: package cheat sheet	36
6	Resources for <code>tidytext</code> (Text Mining with R)	37
6.1	The core reference	37
6.2	If you want to find latent themes: topic modeling (LDA)	37
6.3	If word variants are diluting your counts: <code>stemming</code>	38
6.4	If you want more corpus-linguistics tooling: <code>quanteda</code>	38

6.5	Visualizing results	38
6.6	Cheat sheet	38
6.7	Keep it in scope	39

Welcome

This is the participant manual for the **MBAt Club Datathon 2026** (Econometrics and Business Statistics), held Monday 21 – Tuesday 22 September at Learning and Teaching Building, Clayton campus.

It's a reference to work through **before** the event and to keep open **during** the event — you're not expected to read it cover to cover in one sitting, but every chapter should make sense as a standalone lookup when you're stuck on something specific.

 Never push API keys or credentials to GitHub

Your team's repository is **public**. Your MongoDB connection string (and any other password or API key) must never appear in a committed file — keep it in a local `.Renvi` that stays out of the repo. See Chapter 1 and Chapter 5 for the full setup.

How to use this manual

- Chapter 1 — your team's submission is a public GitHub repository, so this is the one chapter everyone on the team should be comfortable with before Day 1.
- Chapter 2 — how to write and render the reproducible report and the presentation slides your submission requires — both are mandatory, not just for finalists.
- Chapter 3 — the folder layout your team's repository starts from, and how to fill in and render it.
- Chapter 4 and Chapter 5 — the competition dataset lives in a MongoDB Atlas cluster; these two chapters cover the concepts and the hands-on connection guide for pulling it into R.
- Chapter 6 — further reading and techniques for text mining the (mostly free-text) dataset, beyond what's already worked through in the MongoDB chapter.

Before you arrive

- Install R, RStudio (or Positron/VS Code with the R extension), Quarto, and GitHub Desktop.

- Create a GitHub account if you don't already have one, and share your username with organisers — they create and share your team's repository, so you just need to clone it (Chapter 1).
- Skim Chapter 1 and Chapter 2 ahead of time — Day 1 morning is registration, opening remarks, and the dataset introduction, and the working session doesn't start until 10:30 AM, so setup issues are much cheaper to fix beforehand than mid-competition.

Getting help on the day

A help desk is staffed for the full duration of both days. If you're stuck on something in this manual for more than a few minutes, ask rather than losing working time to it.

1 Git + GitHub Desktop

Why this matters

Your submission is a public GitHub repository. A team that can't reliably commit and push loses time to lost work, overwritten files, or a repo that isn't actually up to date at the deadline.

1.1 The daily loop

Everything below is a variation on this:

1. **Pull** the latest changes.
2. Do some work.
3. **Commit** with a short message.
4. **Push** so your team can see it.

1.2 Install (before the event)

1. Install [GitHub Desktop](#).
2. Sign in with a GitHub account (create one first if needed).
3. Let it install Git automatically if prompted.
4. Make sure organisers have your GitHub username so you can be added to your team's repository.

1.3 Get your team's repository

Organisers create and share one repository per team, already set up from the starter template (Chapter 3) — you don't create anything.

1. Get the repo link from organisers.
2. Every team member: GitHub Desktop → **File** → **Clone Repository** → paste the URL → **Clone**.

3. Before the working session starts, confirm you can push: make a trivial commit (e.g. add your name to the README) and push it.

1.4 Work session workflow

1. Click **Fetch origin**, then **Pull origin** if you're behind — do this *before* you edit anything.
2. Edit your files as normal.
3. Check the **Changes** tab — untick anything you didn't mean to include.
4. Write a short summary → **Commit to main**.
5. Click **Push origin**.
6. Repeat: pull → work → commit → push.

Commit often — every working chunk of progress, not just once at the end of the day.

1.5 Avoid stepping on your teammates

- Split work by file/section so two people aren't editing the same lines at once.
- Pull before you start a session; push before you take a break.
- Say out loud (or in chat) before editing a shared file, e.g. the report.

1.6 Fix a merge conflict

1. Open the file GitHub Desktop flags as conflicted.
2. Find the marked sections:

```
<<<<<<< HEAD
your version
=====
their version
>>>>>>> branch-name
```

3. Edit the file to keep the correct content (or merge both).
4. Delete the <<<<<<<, =====, >>>>>>> marker lines.
5. Save, then commit the resolved file as usual.
6. Stuck for more than a few minutes? Ask the help desk.

1.7 Never commit these

- **Credentials** — MongoDB connection string, API keys, `.Renviron` contents. Keep them in a local `.Renviron` file, listed in `.gitignore`.
- **Large raw data dumps** — pull from the database at analysis time instead.
- **Cache/temp folders** — `.quarto/`, `*_cache/`. (`_docs/` is the exception — see Chapter 3: that one *does* need to be pushed, since it's your submitted report and slides.)

If a credential is ever pushed by accident, rotate it immediately — deleting the file does not remove it from the repo's history.

1.8 Prefer the command line?

```
git clone <repo-url>
git status
git add <file>          # or `git add .`
git commit -m "message"
git pull
git push
```

Same repository either way — teammates can mix GitHub Desktop and the terminal freely.

1.9 Before you submit (Day 2, 11:00 AM deadline)

- ☐ The final commit is pushed — an unpushed commit does not count as submitted
- ☐ No credentials appear anywhere in the repo, including old commits
- ☐ Both the rendered HTML and PDF reports open correctly from a fresh clone
- ☐ The presentation slides (Chapter 2) are finished and rendered — required from every team, not just finalists

2 Quarto

i Why this matters

Your submission requires both a reproducible report and a set of presentation slides — Quarto builds both from the same kind of source file. It runs your R code and inserts the real output every time you render, so you never copy-paste a number or a plot in by hand.

The starter template (Chapter 3) already includes both: `docs/report.qmd` for the written report and `docs/slides.qmd` for presentation slides.

2.1 Anatomy of a .qmd file

```
---
title: "Team Name - Datathon Submission"
author: "Team Name"
format: html
---

# Introduction

We analyse the AI Incident Database to answer...

```{r}
library(tidyverse)
summary(cars)
```
```

- Text outside code chunks is normal Markdown.
- Anything inside ````{r} ```` blocks runs at render time.
- Add `#| echo: false` inside a chunk to keep the output but hide the code — useful for staying within a word limit.

2.2 Render your report

- RStudio/Positron: click **Render**, or press Cmd/Ctrl+Shift+K.
- Command line:

```
quarto render report.qmd      # render using the format in the YAML
quarto render report.qmd --to pdf # force a specific format
quarto preview report.qmd     # live preview, reloads on save
```

Use `quarto preview` while you write — it catches broken chunks immediately instead of at the end.

2.3 Set your output format

Both HTML and PDF are required for the report — list both formats in the YAML and Quarto renders one file for each:

```
format:
  html:
    toc: true
    code-fold: true # readers can expand/collapse code blocks
  pdf:
    documentclass: article
```

```
quarto render report.qmd # produces report.html and report.pdf
```

The report has a **3,000-word limit**, excluding code, tables, figure captions, and references.

2.4 Build presentation slides (required from every team)

A finished slide deck is part of your submission, not an optional extra — every team builds one, even though only the 5 finalists actually present it live (2:15–3:30 PM, Day 2: 10 min talk + 5 min Q&A each). Treat it with the same care as the report: it's judged on Professional Communication just like everything else you submit.

```
---
title: "Team Name - Datathon Presentation"
format: revealjs
---
```

1. Start each new slide with a `##` heading.
2. Write R chunks exactly as you would in the report — plots and tables render live into the deck.
3. Render the same way: **Render** button, `Cmd/Ctrl+Shift+K`, or `quarto render slides.qmd`.
4. Open the resulting `.html` in a browser; press `f` for fullscreen, use arrow keys or space to advance.

```
## Introduction

The question we investigated...

## Our Approach

```{r}
ggplot(data, aes(x, y)) + geom_point()
```

## Results

Key finding...
```

2.5 Set up PDF rendering (before the event)

Since PDF is a required output, don't leave this until Day 2. PDF needs LaTeX — install it once, ahead of time:

```
install.packages("tinytex")
tinytex::install_tinytex()
```

Render both formats and open both files before the deadline — a report that renders fine to HTML can still fail on PDF (e.g. an image or table that doesn't fit the page).

2.6 Keep the report reproducible and inside the word limit

1. Render early and often — don't wait until the deadline to find a broken chunk.
2. Decide how much code you want visible in the final report — the `echo`, `warning`, and `message` options (set per chunk, or globally under `execute:` in the YAML) control this. There's no required setup here; choose whatever makes your report read cleanly.

3. Cache slow steps (a MongoDB pull, a model fit) with `#| cache: true` on that chunk.
4. Use `knitr::kable()` or `gt` for tables, and pipe large printed output through `head()` — raw tibble/model printouts can quietly blow out your word count.
5. Keep the raw `.qmd` in your GitHub repo, not just the rendered file — the repo is what gets checked for reproducibility.

2.7 Learn more

- `<starttr.numbat.space>` — the club’s starting point for Quarto.
- <https://quarto.org/docs/guide/> — official docs, every format and option.
- `quarto --help` on the command line, or `?quarto::quarto_render` in R.

Stuck on a YAML typo for more than a few minutes? Ask the help desk.

3 Using the Starter Template

i Why this matters

The template gives every team the same layout and a couple of pieces (a database-connection stub, report/slide skeletons) already wired up, so you spend Day 1 morning on data, not project setup.

3.1 Folder structure

- **src/** — your R script files. Ships with `main.R`, which stubs out a `connect_to_db()` function for you to implement.
- **data/** — intermediate computed datasets you save as you go.
- **docs/** — your Quarto files: `report.qmd` (written report) and `slides.qmd` (presentation) — see Chapter 2.
- **_docs/** — generated automatically when you render (set as the `output-dir` in `_quarto.yml`). This holds your final rendered report and slides — commit and push it, since it's what organisers see.
- **manual/** — reference material from organisers.

Following this layout isn't mandatory, but it's what lets organisers find your work quickly — worth it for the Professional Communication judging pillar.

3.2 Get set up

1. Clone your team's repo (Chapter 1).
2. Open it in RStudio/Positron.
3. Install the packages the template uses: `dplyr`, `mongolite`, [here](#).
4. Open `src/main.R` and implement `connect_to_db()` using your Atlas connection string (Chapter 5) — read it with `Sys.getenv()`, never hardcode it.

3.3 Use your script from a .qmd

Don't copy-paste code between `src/main.R` and your `.qmd` files — source it instead. `docs/report.qmd` already does this at the top:

```
source(here::here("src/main.R"))
```

`here::here()` keeps file paths working no matter which file you render from — always reference files this way rather than relative paths like `"../src/main.R"`.

For more selective imports, `box::use()` is also available if you're already familiar with it — optional, not required.

3.4 Fill in the report and slides

1. `docs/report.qmd` — starts as `format: html` only. Both HTML **and** PDF are required, so add a `pdf:` entry to the YAML before you submit (Chapter 2 shows the format block and PDF setup). Set your team name in the title too.
2. `docs/slides.qmd` — already set to `format: revealjs`. A finished deck is required from every team, not just finalists — only the 5 finalists present it live, but everyone submits one (Chapter 2 covers building slides).

3.5 Render everything

From the project root:

```
quarto render
```

This renders every `.qmd` under `docs/` and writes the output into `_docs/`.

3.6 Submit

There's no file to zip or upload — organisers submit by cloning your repository directly.

1. Run `quarto render` so `_docs/` is up to date.
2. Commit and push everything, including `_docs/` (Chapter 1) — unlike a typical build folder, this one needs to be pushed, since it's your actual submitted report and slides.
3. Confirm the repo is public and the final push landed before the deadline.

Part I

Databases + MongoDB

4 NoSQL and MongoDB: An Introduction for Statisticians

i Why this matters

The competition data lives in MongoDB, not tables. If your mental model is still “rows and joins,” a few of MongoDB’s habits will trip you up — this chapter re-orientates that model before you touch the real connection in Chapter 5.

Already comfortable with SQL vs. NoSQL? Skip straight to Chapter 5 and test your connection before the event starts — see its “Common pitfalls” section on network/allow-listing issues.

Written for readers comfortable with R, data frames, and relational databases (SQL, normalized tables, joins).

4.1 1. Why This Tutorial Exists

If you’ve spent time with relational databases, your mental model of data storage is probably something like this:

- Data lives in **tables** with a **fixed schema** (columns defined in advance, with types).
- Rows relate to each other through **foreign keys**, and you stitch related data back together with **joins**.
- You enforce consistency using constraints (primary keys, foreign keys, NOT NULL, etc.).
- A single logical “thing” — like a customer with multiple orders — is normalized across several tables to avoid duplication.

This model maps naturally onto how you already think about data as a statistician: a rectangular data frame, one row per observation, one column per variable, `merge()` or `dplyr::left_join()` to combine data frames.

NoSQL databases (of which MongoDB is the most popular example) relax or abandon several of these assumptions. Understanding *why* they do this, and *what you gain and lose*, is the goal of this tutorial.

4.2 2. What “NoSQL” Actually Means

“NoSQL” is a loose umbrella term (sometimes glossed as “Not Only SQL”) covering several different data models that don’t follow the relational (table-and-join) paradigm. The major families are:

| Type | Data model | Example systems |
|-------------|---------------------------------------|------------------|
| Document | Semi-structured documents (JSON-like) | MongoDB, CouchDB |
| Key-value | Simple key → value lookups | Redis, DynamoDB |
| Wide-column | Sparse, column-family tables | Cassandra, HBase |
| Graph | Nodes and edges | Neo4j, ArangoDB |

This tutorial focuses on **document stores**, and specifically **MongoDB**, because they’re the most common entry point and the comparison to relational tables is the most direct.

4.3 3. The Core Idea: Documents Instead of Rows

 The data used in this example

The examples used in this page are not reflective of the actual data you will use in the competition! They are only presented to get an understanding of the functionalities.

In a relational database, a customer and their orders might be split across two tables:

```
-- customers table
customer_id	name	email
1           | Alice  | alice@email.com

-- orders table
order_id	customer_id	item	amount
101      | 1           | Widget | 9.99
102      | 1           | Gadget | 19.99
```

To get “Alice and all her orders,” you JOIN on `customer_id`.

In MongoDB, you’d likely store this as a **single document**:

```
{
  "_id": "1",
  "name": "Alice",
  "email": "alice@email.com",
  "orders": [
    { "item": "Widget", "amount": 9.99 },
    { "item": "Gadget", "amount": 19.99 }
  ]
}
```

A “document” is essentially a JSON object (technically stored as **BSON**, a binary superset of JSON with extra types like dates and integers). Documents are grouped into **collections**, which are the loose analogue of tables — except a collection has no fixed schema. Two documents in the same collection can have completely different fields.

4.3.1 A translation table for vocabulary

| Relational term | MongoDB term |
|-----------------|---|
| Database | Database |
| Table | Collection |
| Row | Document |
| Column | Field |
| Primary key | <code>_id</code> field |
| Join | Embedding, or <code>\$lookup</code> (an actual join-like operation) |
| Schema (rigid) | Schema (optional / flexible) |

4.4 4. Why Embed Instead of Normalize?

This is the conceptual heart of the difference, and it’s worth dwelling on because it inverts a habit that’s deeply ingrained if you’ve learned relational design.

Relational design philosophy: normalize aggressively. Store each fact once. Avoid redundancy. Joins are cheap(-ish) and consistency matters more than read speed.

Document design philosophy: model around your *access patterns*. If you almost always fetch a customer along with their orders together, store them together, even if that means the word “Widget” is duplicated across many documents. Reads become a single lookup instead of a join across multiple tables — this matters a lot for web-application workloads with high read volume and simple, predictable query shapes.

The tradeoff: - **Relational:** harder to scale horizontally, but strong guarantees around consistency and no duplicated data to keep in sync. - **Document:** easier to scale horizontally and often faster for “fetch this one entity and everything about it” queries, but you may duplicate data and must manage the consequences (e.g., if Alice changes her email, and it’s embedded in old order documents, you may need to update it in multiple places, or decide you don’t care).

There’s no free lunch here — it’s an explicit modeling decision, not a strictly-better technology.

4.5 5. Schema Flexibility: A Blessing and a Curse

In a relational database, adding a column to a 10-million-row table requires an `ALTER TABLE` migration and some care. In MongoDB, you can insert a document with a new field at any time — the collection doesn’t complain.

For a statistician, the closest analogy is the difference between:

- An R `data.frame`/tibble with fixed columns (relational table), versus
- A `list` of `lists`, where each element can have different named entries (MongoDB collection).

```
# Relational-style: a data.frame, fixed columns
df <- data.frame(id = 1:2, name = c("Alice", "Bob"), age = c(30, NA))

# Document-style: a list of lists, heterogeneous structure
docs <- list(
  list(id = 1, name = "Alice", age = 30, hobbies = c("chess", "running")),
  list(id = 2, name = "Bob") # no age, no hobbies - and that's fine
)
```

This flexibility is genuinely useful when your data is naturally irregular (user-generated content, event logs with varying attributes, nested/hierarchical records). It’s a liability when you *want* the enforcement — e.g., you want the database itself to guarantee that `age` is always numeric, or that every document has an `email`. MongoDB does support optional **schema validation** rules, but they’re opt-in rather than the default posture of the system.

4.6 6. Querying: dplyr/SQL Habits vs. MongoDB's Query Language

MongoDB queries are expressed as documents themselves (essentially, nested lists/JSON), which feels unusual at first if you're used to SQL's `SELECT ... WHERE ...` syntax or `dplyr::filter()`.

4.6.1 Comparison table

| Task | SQL | dplyr | MongoDB (shell syntax) |
|-------------------|--|--|---|
| Filter rows | <code>SELECT * FROM orders WHERE amount > 10</code> | <code>filter(orders, amount > 10)</code> | <code>db.orders.find({ amount: { \$gt: 10 } })</code> |
| Select columns | <code>SELECT item, amount FROM orders</code> | <code>select(orders, item, amount)</code> | <code>db.orders.find({}, { item: 1, amount: 1 })</code> |
| Sort | <code>ORDER BY amount DESC</code> | <code>arrange(desc(amount))</code> | <code>.sort({ amount: -1 })</code> |
| Group + aggregate | <code>GROUP BY customer_id</code> | <code>group_by(customer_id) %>% summarise(...)</code> | <code>\$group</code> stage in an aggregation pipeline |

MongoDB's analogue to a SQL query with `GROUP BY`/joins/multiple transformations is the **aggregation pipeline** — a sequence of stages, each transforming the documents flowing through it, conceptually similar to chaining `dplyr` verbs with `%>%`:

```
db.orders.aggregate([
  { $match: { amount: { $gt: 5 } } },      // like filter()
  { $group: { _id: "$customer_id",
              total: { $sum: "$amount" } } }, // like group_by() %>% summarise()
  { $sort: { total: -1 } }                 // like arrange(desc())
])
```

If you're comfortable reading a `dplyr` pipe, the aggregation pipeline will feel more familiar than it first appears — it's a sequence of verbs applied in order, just expressed as JSON stage objects instead of R function calls.

4.7 7. Using MongoDB from R

You don't have to leave R to work with MongoDB. The `mongolite` package provides a clean interface, and the workflow will feel close to what you already do with DBI.

```
install.packages("mongolite")
library(mongolite)

# Connect to a collection (analogous to opening a DBI connection to a table)
orders <- mongo(collection = "orders", db = "shop", url = "mongodb://localhost")

# Insert documents (rows) - takes a data.frame directly
orders$insert(data.frame(item = "Widget", amount = 9.99, customer_id = 1))

# Query and get a data.frame back - this is the key convenience
result <- orders$find('{"amount": {"$gt": 5}}')
head(result)

# Aggregation pipeline, expressed as a JSON string
pipeline <- '[
  {"$group": {"_id": "$customer_id", "total": {"$sum": "$amount"}}},
  {"$sort": {"total": -1}}
]'
summary_df <- orders$aggregate(pipeline)
```

Compare this to a typical relational workflow with DBI:

```
library(DBI)
library(RPostgres)

con <- dbConnect(RPostgres::Postgres(), dbname = "shop")
result <- dbGetQuery(con, "SELECT * FROM orders WHERE amount > 5")
```

The big practical difference: `mongolite` queries and pipeline stages are written as **JSON strings**, not SQL strings — but the round trip (query in, tibble/data.frame out) is conceptually identical, which is good news for how much of your existing R workflow transfers.

4.8 8. Consistency, Transactions, and Why This Matters to You

Relational databases are traditionally built around **ACID** guarantees (Atomicity, Consistency, Isolation, Durability) — a transaction either fully happens or fully doesn't, and the database enforces referential integrity via constraints.

Historically, NoSQL document stores traded some of these guarantees for scalability, often described (loosely) via the **CAP theorem**: in the presence of a network partition across distributed servers, a system must choose between **Consistency** (every read gets the latest write) and **Availability** (every request gets *a* response, possibly stale). Many early NoSQL systems leaned toward availability and “eventual consistency.”

Modern MongoDB has narrowed this gap substantially — it now supports multi-document ACID transactions — but the *default* mode of use, and the ecosystem's design instincts, still favor flexible, denormalized, single-document operations over enforced cross-table integrity. If your work involves financial ledgers or anything where “the numbers must always add up, no exceptions, ever,” that's traditionally been relational-database territory, and it's worth understanding that this is a design lineage question, not just a feature checkbox — you can bolt strong consistency onto MongoDB, but it's swimming against the current of how the system is typically used.

4.9 9. When Each Approach Tends to Fit Better

| Favor relational (SQL) when... | Favor document (MongoDB) when... |
|--|---|
| Your data is naturally tabular and stable in structure | Your data is nested, hierarchical, or has varying fields per record |
| You need complex multi-table joins and strong referential integrity | You mostly fetch one entity plus its closely related data together |
| Strong consistency/transactions across entities is essential | High write/read throughput and horizontal scaling matter more |
| Your analysis workflow is fundamentally about tidy rectangular tables (which is common in statistics!) | Your data is closer to logs, events, user profiles, or content with irregular shape |

For a lot of statistical work — clean, rectangular datasets destined for modeling — relational databases (or just flat files/data frames) remain a very natural fit, and you'll often find yourself **flattening** MongoDB documents back into rectangular data frames before doing any actual analysis, since most statistical methods assume tidy, rectangular input.

4.10 10. A Worked Mini-Example: Same Data, Both Ways

Relational (two tables + a join):

```
CREATE TABLE customers (customer_id INT PRIMARY KEY, name TEXT);
CREATE TABLE orders (
  order_id INT PRIMARY KEY,
  customer_id INT REFERENCES customers(customer_id),
  item TEXT,
  amount NUMERIC
);

SELECT c.name, o.item, o.amount
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id;
```

Document (one collection, embedded):

```
library(mongolite)
customers <- mongo("customers", db = "shop")

customers$insert('{
  "_id": 1,
  "name": "Alice",
  "orders": [
    {"item": "Widget", "amount": 9.99},
    {"item": "Gadget", "amount": 19.99}
  ]
}')

# Pull it back out, then flatten to a data.frame for analysis
raw <- customers$find()
flat <- tidyr::unnest(raw, orders)
```

Notice the last line: even when working with MongoDB, you often end up reaching for `tidyr::unnest()` to get back to the rectangular shape you actually want for modeling. This is a common and completely normal pattern — document stores are often optimized for how an *application* writes and reads data, not for how a *statistician* wants to analyze it, and translating between the two is a routine part of the workflow.

4.11 11. Summary

- MongoDB stores flexible, JSON-like **documents** in **collections**, instead of enforcing rows-in-tables with a fixed schema.
- The relational instinct to *normalize* is replaced by a document instinct to *denormalize/embed*, optimized around how the data will be read.
- Querying feels different syntactically (JSON-shaped queries, aggregation pipelines) but maps conceptually onto SQL/`dplyr` verbs you already know.
- Consistency and transactional guarantees are available in modern MongoDB but aren't the system's traditional default posture the way they are in relational databases.
- For statistical analysis specifically, you'll frequently pull data *out* of MongoDB and flatten it back into rectangular form — so document databases are often best understood as an operational/application-layer tool, with relational-style rectangular data as the destination for actual modeling.

If you want to go deeper next, worthwhile follow-ups are: MongoDB's aggregation pipeline in more depth, schema validation rules, and how sharding/replication work for horizontal scaling.

5 Getting Your Competition Data Out of MongoDB Atlas and Into R

Why this matters

You can't analyse the dataset until you can pull it out of Atlas and into a tidy data frame. Working through this before the event means the connection and package setup aren't things you're debugging on the day.

Tip

The starter template's `src/main.R` (Chapter 3) has an empty `connect_to_db()` function waiting for you — that's where the connection code below goes.

This sheet walks you through connecting to the competition's MongoDB Atlas cluster from R, pulling the data down into a tidy data frame, and doing text analysis on it with the tidyverse / tidytext ecosystem. It's written against the actual schema you've been given (this is the AI Incident Database schema), so the field names below match what you'll actually see. Work through it top to bottom before the competition starts so the connection and package setup is one less thing to debug on the day.

5.1 1. What you'll need

- R (4.x) and RStudio (or Positron / VS Code with the R extension)
- The connection string for your Atlas cluster (competition organisers will give you this , it looks like `mongodb+srv://<username>:<password>@ai-incident-database.ugkyb5n.mongodb.net/`)
- The database name is 'aiidprod'

Install the packages you'll need once, up front:

```
install.packages(c(
  "mongolite",    # connect to MongoDB and pull data into R
  "tidyverse",    # dplyr, purrr, ggplot2, tidyr, etc.
  "tidytext",     # tokenisation, sentiment lexicons, tf-idf
  "jsonlite",     # handling nested JSON/BSON fields
  "textclean",    # cleaning up messy text (contractions, HTML, etc.)
  "wordcloud2",   # optional, for word cloud visuals
  "widyR"         # optional, for word-pair / correlation analysis
))
```

`mongolite` is the key package here, it's the simplest way to talk to MongoDB from R and it hands data back as a data frame, which is exactly the shape tidyverse tools expect.

5.2 2. Know your collections before you connect

The schema you were given defines several collections. The two you'll care about for text analysis are:

- **reports** — the actual documents/articles. This is where the free text lives. Key fields:
 - `_id` (objectId), `report_number` (number) — `report_number` is the human-friendly key used to link a report to the incident(s) it belongs to
 - `title`, `text`, `plain_text` — `text` may retain markdown/HTML formatting from the original source; `plain_text` is the cleaned version and is almost always what you want for tokenising/sentiment work
 - `description` — a short editor-written summary, separate from the report body
 - `authors`, `submitters` — arrays of strings
 - `tags` — array of free-text tags
 - `language`, `source_domain`, `url`
 - `date_published`, `date_submitted`, `date_downloaded`, `date_modified`
 - `embedding` — a precomputed vector embedding of the text (nested object with a `vector` field); you generally won't need to touch this unless you're doing similarity/clustering work directly on the provided vectors
 - `is_incident_report` (boolean)
- **incidents** — the higher-level “event” that one or more reports are describing. Key fields:
 - `_id` (objectId), `incident_id` (number)
 - `title`, `description`, `date`

- **reports** — an **array of report_number values** (not full documents) pointing back into the **reports** collection — this is how you join the two collections
- **Alleged deployer of AI system, Alleged developer of AI system, Alleged harmed or nearly harmed parties** — arrays of entity-name strings
- **implicated_systems** — array of strings
- **editors, editor_notes**
- **classifications** — tagging/label data (an **attributes** field) that links back to **incidents/reports** by their IDs — useful if you want to bring in categorical labels alongside the report text.
- **entities** — a lookup table of named entities (**entity_id, name**) referenced by the “alleged deployer/developer/harmed parties” fields above.

For most text-analysis tasks, you’ll mainly work with **reports.plain_text**, optionally joined to **incidents** for grouping/labels.

5.3 3. Connect to the cluster

Never hard-code your username and password directly into a script you might commit to GitHub or share with teammates. Use an **.Renviron** file instead.

Create/edit your **.Renviron** file (run this once):

```
usethis::edit_r_environ()
```

Add a line like this, save, and restart R:

```
MONGO_URI="mongodb+srv://myusername:mypassword@ai-incident-database.ugkyb5n.mongodb.net/"
```

Now **connect from your script** — one connection object per collection:

```
library(mongolite)

db_name <- "aiidprod"

reports_conn <- mongo(collection = "reports", db = db_name,
                      url = Sys.getenv("MONGO_URI"))
```

```

incidents_conn <- mongo(collection = "incidents", db = db_name,
                        url = Sys.getenv("MONGO_URI"))

# quick sanity check
reports_conn$count()
incidents_conn$count()

```

If `$count()` returns a number rather than an error, you're connected.

5.4 4. Pulling data out

5.4.1 4.1 Grab the fields you actually need

Don't pull every field (especially not `embedding`, which is a large numeric vector per document) if you don't need it. Use the `fields` argument to project server-side:

```

library(tidyverse)

report_cols <- '{
  "_id": 1, "report_number": 1, "title": 1, "plain_text": 1,
  "description": 1, "tags": 1, "authors": 1, "language": 1,
  "source_domain": 1, "date_published": 1
}'

reports <- reports_conn$find(
  fields = report_cols) |> as_tibble()

glimpse(reports)

```

5.4.2 4.2 Filter server-side

`mongolite` accepts standard MongoDB query syntax as a JSON string, e.g. to grab only English-language reports:

```
reports_en <- reports_conn$find(
  query = '{"language": "en"}',
  fields = '{"_id": 1, "report_number": 1, "title": 1, "plain_text": 1}'
) |> as_tibble()
```

For more information on the query syntax visit, <https://jeroen.github.io/mongolite/query-data.html#query-syntax>

5.4.3 4.3 Pull the incidents and join on report_number

`incidents$reports` is an array of `report_numbers`, not embedded report documents, so joining means unnesting that array and matching it up:

```
incidents <- incidents_conn$find(
  fields = '{
    "_id": 1, "incident_id": 1, "title": 1, "description": 1, "date": 1,
    "reports": 1, "Alleged deployer of AI system": 1,
    "Alleged developer of AI system": 1,
    "Alleged harmed or nearly harmed parties": 1
  }'
) |> as_tibble()

# one row per (incident_id, report_number) pair
incident_report_map <- incidents |>
  select(incident_id, reports) |>
  unnest_longer(reports) |>
  rename(report_number = reports)

# now every report row can carry its incident's metadata
reports_with_incident <- reports |>
  left_join(incident_report_map, by = "report_number") |>
  left_join(
    incidents |> select(incident_id, incident_title = title, date),
    by = "incident_id"
  )
```

5.4.4 4.4 Aggregation pipelines (optional, for heavier lifting)

For grouping/counting server-side before pulling data down:

```
reports_per_year <- reports_conn$aggregate('[
  { "$group": {
    "_id": { "$substr": ["$date_published", 0, 4] },
    "n": { "$sum": 1 }
  }},
  { "$sort": { "_id": 1 } }
]') |> as_tibble()
```

5.4.5 4.5 Other nested/array fields

Several fields come back from `mongolite` as **list-columns**: `authors`, `submitters`, `tags` (on `reports`); the “alleged deployer/developer/harmed parties” fields (on `incidents`). Each of these is a simple *array of strings per document*, so `unnest_longer()` is usually what you want (not `unnest_wider()`, which is for nested *objects*):

```
reports_authors <- reports |>
  select(report_number, authors) |>
  unnest_longer(authors) # one row per (report_number, author)

incidents_parties <- incidents |>
  select(incident_id, `Alleged harmed or nearly harmed parties`) |>
  unnest_longer(`Alleged harmed or nearly harmed parties`) |>
  rename(harmed_party = `Alleged harmed or nearly harmed parties`)
```

Guard against missing/empty text before tokenising:

```
reports <- reports |>
  mutate(plain_text = replace_na(plain_text, ""))
```

5.5 5. Cleaning the text

`plain_text` is already stripped of most markup, but scraped news text still tends to carry boilerplate (bylines, “read more” links, share-button text). Do a light cleaning pass and keep the original column in case later analysis needs it:

Warning

The following is only a suggestion and is not guaranteed to produce fully cleaned text ready for analysis.

```
library(textclean)

reports <- reports |>
  mutate(
    text_clean = plain_text |>
      replace_contraction() |>          # "don't" -> "do not"
      replace_url() |>
      replace_html() |>
      str_squish()                      # trim/collapse whitespace
  )
```

5.6 6. Tokenising with tidytext

tidytext works on the “one-token-per-row” principle, which plugs directly into dplyr/ggplot2.

```
library(tidytext)

tokens <- reports |>
  select(report_number, text_clean) |>
  unnest_tokens(word, text_clean)

# remove stop words
tokens_clean <- tokens |>
  anti_join(stop_words, by = "word")
```

5.6.1 6.1 Word frequencies

```
top_words <- tokens_clean |>
  count(word, sort = TRUE)
```

```
top_words |>
  slice_max(n, n = 20) |>
  ggplot(aes(x = reorder(word, n), y = n)) +
  geom_col() +
  coord_flip() +
  labs(x = NULL, y = "count", title = "Most frequent words across reports")
```

5.6.2 6.2 TF-IDF (which words distinguish one incident/domain from another)

This is a natural fit here: use `incident_id` or `source_domain` as the grouping variable to see what language is distinctive to a given incident or outlet.

```
tfidf <- reports_with_incident |>
  select(report_number, incident_id, text_clean) |>
  unnest_tokens(word, text_clean) |>
  anti_join(stop_words, by = "word") |>
  count(incident_id, word, sort = TRUE) |>
  bind_tf_idf(word, incident_id, n)

tfidf |>
  group_by(incident_id) |>
  slice_max(tf_idf, n = 10) |>
  ungroup()
```

5.6.3 6.3 N-grams

For phrase-level patterns rather than single words:

```
bigrams <- reports |>
  unnest_tokens(bigram, text_clean, token = "ngrams", n = 2) |>
  separate(bigram, into = c("word1", "word2"), sep = " ") |>
  filter(!word1 %in% stop_words$word, !word2 %in% stop_words$word) |>
  count(word1, word2, sort = TRUE)
```

5.7 7. Sentiment analysis

`tidytext` bundles several sentiment lexicons (`bing`, `afinn`, `nrc`, `loughran`). Join tokens to a lexicon the same way you joined stop words. `loughran` is worth knowing about here specifically — it was built for financial/news text and includes categories like “uncertainty” and “litigious”, which can be more informative than generic positive/negative for news-style incident reports.

```
sentiment_scores <- tokens_clean |>
  inner_join(get_sentiments("bing"), by = "word") |>
  count(report_number, sentiment) |>
  pivot_wider(names_from = sentiment, values_from = n, values_fill = 0) |>
  mutate(net_sentiment = positive - negative)
```

For a numeric score per word (rather than positive/negative labels), use `afinn`:

```
afinn_scores <- tokens_clean |>
  inner_join(get_sentiments("afinn"), by = "word") |>
  group_by(report_number) |>
  summarise(sentiment_score = sum(value), .groups = "drop")
```

`get_sentiments()` will prompt a one-time download/license confirmation the first time you use a given lexicon — do this well before the competition starts, not on the day, in case you don’t have internet access at the venue.

5.8 8. Word co-occurrence / correlation (optional, `widyr`)

Useful for exploring which words tend to appear together within the same report:

```
library(widyr)

word_pairs <- tokens_clean |>
  pairwise_count(word, report_number, sort = TRUE)

word_cors <- tokens_clean |>
  group_by(word) |>
  filter(n() >= 20) |> # drop very rare words first
  pairwise_cor(word, report_number, sort = TRUE)
```

5.9 9. Putting it together: a minimal end-to-end script

```
library(mongolite)
library(tidyverse)
library(tidytext)
library(textclean)

db_name <- "your_database_name"
reports_conn <- mongo(collection = "reports", db = db_name,
                      url = Sys.getenv("MONGO_URI"))

reports <- reports_conn$find(
  fields = '{"_id": 1, "report_number": 1, "title": 1, "plain_text": 1, "tags": 1}'
) |>
  as_tibble() |>
  mutate(
    plain_text = replace_na(plain_text, ""),
    text_clean = str_squish(replace_contraction(replace_url(plain_text, "")))
  )

word_counts <- reports |>
  unnest_tokens(word, text_clean) |>
  anti_join(stop_words, by = "word") |>
  count(word, sort = TRUE)

word_counts |> slice_max(n, n = 15)
```

5.10 10. Common pitfalls

- **Connection refused / timeout** — your IP isn't allow-listed in Atlas Network Access, or you're on a network (uni wifi, VPN) that blocks the MongoDB port. Test the connection *before* the competition.
- **Pulling embedding by accident** — it's a full numeric vector per document; leave it out of your `fields` projection unless you specifically need it, or your `find()` calls will be slow and memory-heavy.
- **Using `text` instead of `plain_text`** — `text` can retain markdown/HTML from the original source, which will pollute your tokens (`<`, `##`, `href=`, etc.). Prefer `plain_text` for tokenising.

- **unnest_wider()** on the wrong field — authors, tags, and the “alleged parties” fields are arrays of *strings*, so they need `unnest_longer()`. `unnest_wider()` is for fields that are nested *objects* (there aren’t many of those in `reports/incidents`).
- **Joining incidents to reports on the wrong key** — `incidents.reports` holds `report_number` values, not `_id` (objectId) values. Join on `report_number`, not `_id`.
- **Encoding issues (garbled characters)** — check `Encoding()` on your text column and re-encode with `iconv()` if you see mojibake.
- **Credentials in scripts** — always use `Sys.getenv()` against an `.Renv` file, never paste the connection string directly into a script you’ll share or commit.

5.11 11. Quick reference: package cheat sheet

| Task | Package | Key function(s) |
|---------------------------|------------------|---|
| Connect to MongoDB | mongolite | <code>mongo()</code> , <code>\$find()</code> , <code>\$aggregate()</code> , <code>\$count()</code> |
| Wrangle data | dplyr /
tidyr | <code>mutate()</code> , <code>filter()</code> , <code>unnest_longer()</code> , <code>left_join()</code> |
| Clean text | textclean | <code>replace_contraction()</code> , <code>replace_url()</code> ,
<code>replace_html()</code> |
| Tokenise | tidytext | <code>unnest_tokens()</code> |
| Frequency / TF-IDF | tidytext | <code>count()</code> , <code>bind_tf_idf()</code> |
| Sentiment | tidytext | <code>get_sentiments()</code> , <code>inner_join()</code> |
| Word
pairs/correlation | widyr | <code>pairwise_count()</code> , <code>pairwise_cor()</code> |
| Plotting | ggplot2 | <code>geom_col()</code> , <code>geom_point()</code> |

Good luck with the competition.

6 Resources for tidytext (Text Mining with R)

i Why this matters

The dataset is mostly free text. Chapter 5 already walks through tokenising, TF-IDF, sentiment, and word co-occurrence — this chapter is where to go for more depth or techniques it doesn't cover.

6.1 The core reference

- [Text Mining with R](#) by Julia Silge and David Robinson (free, by the tidytext package authors). Relevant chapters:
 - Ch. 1 — the tidy text format (`unnest_tokens()`)
 - Ch. 2 — sentiment analysis
 - Ch. 3 — word/document frequency (tf-idf)
 - Ch. 4 — n-grams and word correlations
 - Ch. 6 — topic modeling
- `?tidytext::unnest_tokens`, `?tidytext::bind_tf_idf`, or `vignette(package = "tidytext")` for the function reference.

6.2 If you want to find latent themes: topic modeling (LDA)

```
library(topicmodels)

dtm <- tokens_clean |>
  count(report_number, word) |>
  cast_dtm(report_number, word, n)

lda_model <- LDA(dtm, k = 5, control = list(seed = 1234))

tidy(lda_model, matrix = "beta") |> # word probabilities per topic
```

```
group_by(topic) |>
slice_max(beta, n = 10)
```

`tidy()` converts the model back into a tidy data frame so the rest of your `dplyr/ggplot2` pipeline still applies. See Text Mining with R, Ch. 6.

6.3 If word variants are diluting your counts: stemming

```
library(SnowballC)

tokens_clean <- tokens_clean |>
  mutate(word_stem = wordStem(word))
```

Use deliberately, not automatically — stemming can hurt readability in a report meant for human judges.

6.4 If you want more corpus-linguistics tooling: quanteda

[quanteda](#) is an alternative to `tidytext`, with built-in keyword-in-context (`kwic()`) and document-feature matrix tools. Pick one framework and stay consistent through your analysis.

6.5 Visualizing results

- `ggplot2::geom_col()` / `geom_point()` for frequency and tf-idf charts.
- `wordcloud2` for a quick visual summary — pair with an actual frequency table, since word clouds are weak for precise comparisons.
- `ggraph` + `igraph` on `widyr::pairwise_cor()` output for word-correlation networks.

6.6 Cheat sheet

Posit's [tidytext cheat sheet](#) — one page, covers tokenisation, sentiment, tf-idf. Keep it open during the event.

6.7 Keep it in scope

Text mining is a means to an analytical insight, not the insight itself. A clear tf-idf or sentiment analysis tied to a specific question scores better than a kitchen-sink pipeline with no narrative tying it together.